

SecurityAudit

KIRAPAY Deal (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	9
Intended Smart Contract Functions	10
Code Quality	12
Audit Resources	12
Dependencies	13
Severity Definitions	13
Status Definitions	14
Audit Findings	15
Conclusion	40
Our Methodology	41
Disclaimers	43
About Hashlock	44

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The KIRAPAY Deal team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

KIRAPAY is a Web3 fintech infrastructure platform focused on cross-chain crypto payments. The project provides a non-custodial payment gateway that allows merchants, developers, and online businesses to accept payments from multiple blockchains and tokens while settling funds in a preferred token or chain. Its infrastructure abstracts away the complexity of bridges, swaps, and gas fees through an intent-based payment system, aiming to make crypto payments as seamless as traditional online checkout experiences. The platform also offers APIs, SDKs, payment links, and merchant tools for integrating crypto payments into e-commerce, SaaS, creator platforms, and decentralized applications.

Project Name: KIRAPAY Deal

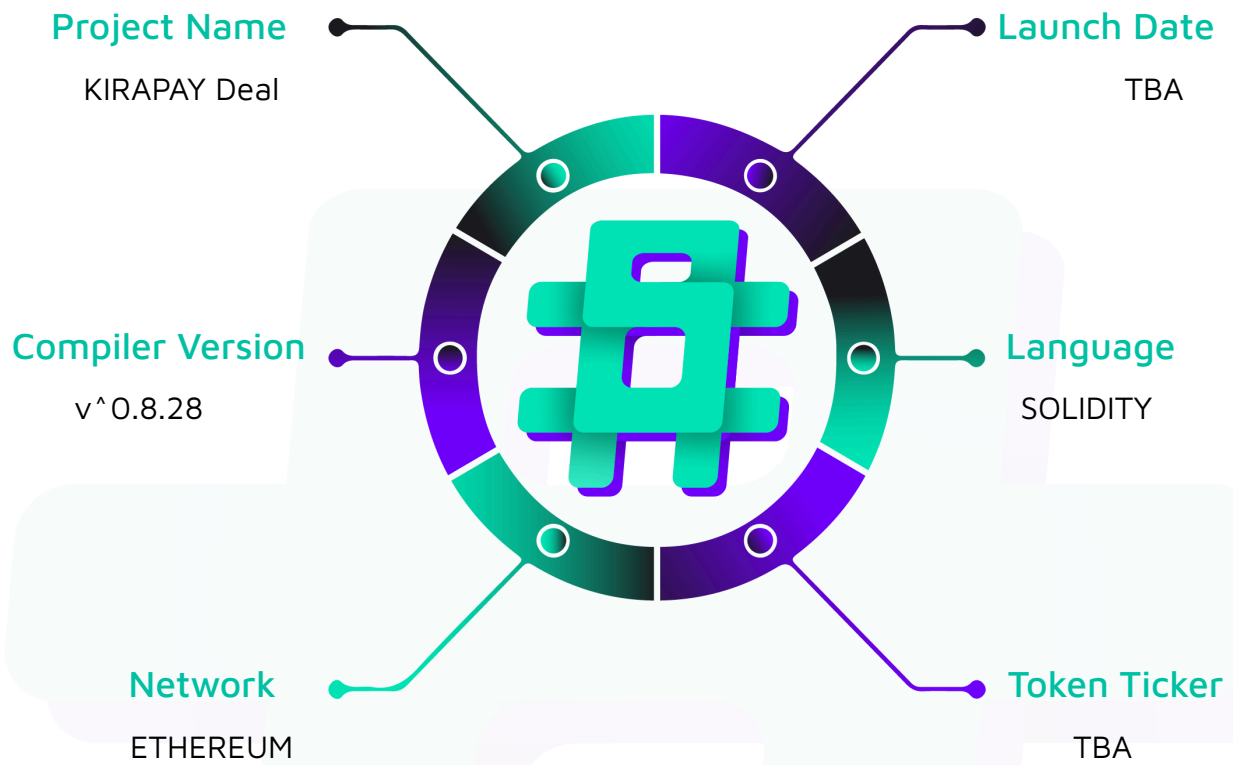
Project Type: Defi

Compiler Version: ^0.8.28

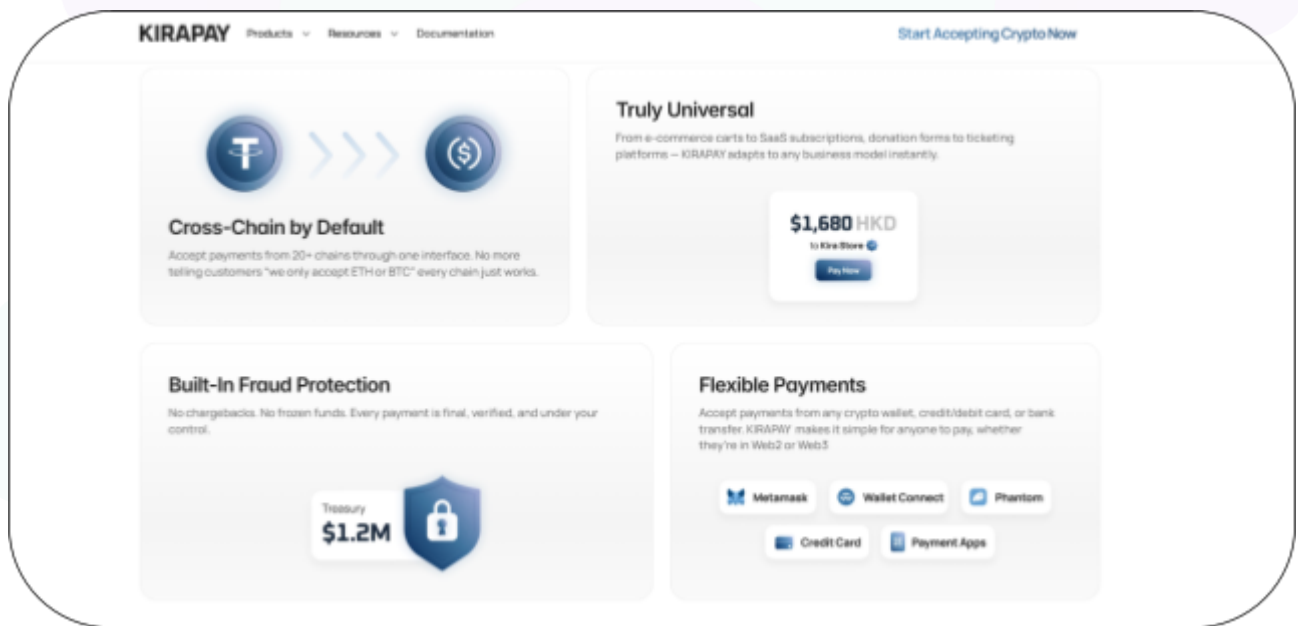
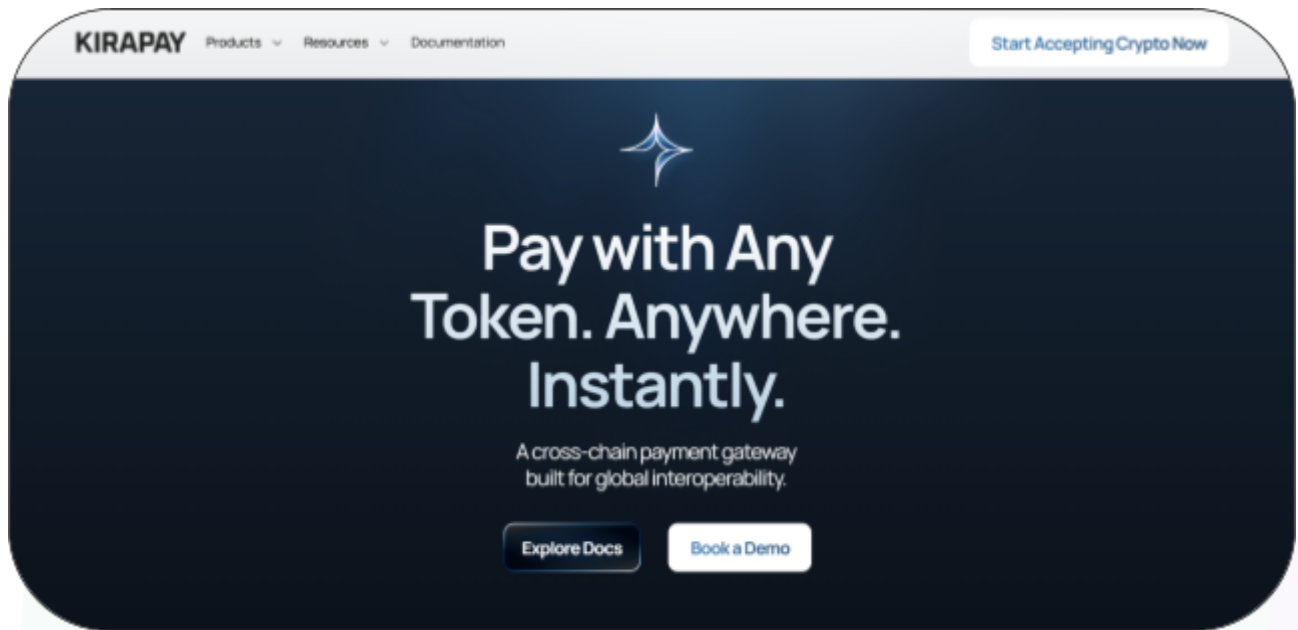
Website: <https://www.kira-pay.com/>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the KIRAPAY Deal project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	KIRAPAY Deal Smart Contracts
Network	Ethereum
Language	Solidity
Audit Date	May, 2026
Contract 1	V3SpokePoolInterface.sol
Contract 2	KiraPayTypes.sol
Contract 3	KiraGatewayLib.sol
Contract 4	TokenUtils.sol
Contract 5	SwapMath.sol
Contract 6	KiraPayGateway.sol
Contract 7	KiraBridgeLanding.sol
Contract 8	KiraReusableVault.sol
Contract 9	KiraAcrossBridger.sol
Contract 10	KiraSwapExecutor.sol
Contract 11	ProtocolFeeManager.sol
Contract 12	KiraPayPricer.sol
Contract 13	KiraInvoice.sol
Contract 14	KiraBridger.sol
Contract 15	KiraAccountRegistry.sol
Contract 16	KiraInvoiceFactory.sol

Contract 17	KiraReusableVaultFactory.sol
Audited GitHub Commit Hash	c193f3ab15403799276aeb411639d5511c77b03fd
Re-audit GitHub Commit Hash	b8dd2d906f968c6442a166b73e39a494c7eda457
Fix Review GitHub Commit Hash	31f4c25976b6df816437e22eae0c5d234dc60464

Additional Scope Note:

During the fix review phase, the client submitted an additional in-scope design change related to the updated fee model. Hashlock reviewed this change and the audit commit is reflected in the "Re-audit GitHub Commit Hash".

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 High severity vulnerability

7 Medium severity vulnerabilities

4 Low severity vulnerabilities

1 QA

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Note: Finding H-01 was added to this report following an additional in-scope fee model change submitted during the fix review phase. The issue was identified as part of that expanded review scope and has therefore been included after the first review in this version of the report.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
KiraPayGateway.sol Central orchestrator for invoice and reusable-vault payment flows (process, claim, fast-finish, bridge, refund).	Contract achieves this functionality.
KiraBridgeLanding.sol Ephemeral per-transfer landing contract that holds bridged tokens until the gateway sweeps them.	Contract achieves this functionality.
KiraReusableVault.sol Minimal EIP-1167 clone that holds payer funds for a reusable vault until the gateway sweeps them.	Contract achieves this functionality.
KiraAcrossBridger.sol Across SpokePool adapter that implements outbound bridging for KiraBridger on a given chain.	Contract achieves this functionality.
KiraSwapExecutor.sol Gateway-only executor that runs whitelisted DEX swaps and final payment-handler calls.	Contract achieves this functionality.
ProtocolFeeManager.sol Ownable fee registry that computes protocol fees per account and forwards collected tokens to a collector.	Contract achieves this functionality.
KiraPayPricer.sol Immutable oracle helper that verifies signed USD price data and rejects stale quotes.	Contract achieves this functionality.
KiraInvoice.sol	Contract achieves this

Minimal EIP-1167 clone that holds payer funds for a single invoice until the gateway sweeps them.	functionality.
KiraBridger.sol Ownable outbound bridge router that delegates sendToChain to per-chain IKiraBridger adapters.	Contract achieves this functionality.
KiraAccountRegistry.sol UUPS-upgradeable registry of onboarded accounts, statuses, and per-account fee overrides.	Contract achieves this functionality.
KiraReusableVaultFactory.sol Deploys deterministic CREATE2 KiraReusableVault clones from ReusableVaultParams.	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the KIRAPAY Deal project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the KIRAPAY Deal project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-severity issues should be resolved before deployment. While they do not typically lead to a complete loss of funds, they may result in partial loss of funds or unintended behavior under certain conditions.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] KiraGatewayLib#collectFeeFromGateway - Unbounded overheadFee lets a relayer drain the merchant settlement

Description

OverheadFee is a relayer supplied calldata field with no meaningful on-chain bound, allowing a relayer to claim nearly the entire settlement amount as overhead.

Vulnerability Details

CollectFeeFromGateway deducts overheadFee from the settlement before computing the protocol fee:

```
require(overheadFee < amountBeforeFee, IKiraPayGateway.FeeTooHigh());

uint256 protocolFee = cfg.feeManager.computeFee(amountBeforeFee - overheadFee, isRouted,
accountId);

uint256 fee = protocolFee + overheadFee;
```

A relayer settling an invoice for toAmount = 1_000e6 USDC can pass overheadFee = 999_999_999. This leaves computeFee(1, ...) ≈ 0 in protocol fees, so fee = 999_999_999 is transferred to the fee manager while the merchant receives 1 wei.

All six settlement entry points funnel through this function (processInvoice, fastFinishInvoice, claimInvoice, processVaultTransfer, fastFinishVaultTransfer, claimVaultTransfer), and the same unbounded check is replicated in KiraSwapExecutor._collectFeeFromExecutor (KiraSwapExecutor.sol:243) for the swap path.

Impact

Any authorized relayer can divert nearly the full settlement amount away from the merchant on every invoice and vault transfer.



Recommendation

Consider at least one of the following approaches:

1. Add a `maxOverheadBps` to the fee configuration and cap overhead relative to `amountBeforeFee`.
2. Add a `maxOverheadFee` field to both attestation typehashes so the off-chain signer commits to a per-settlement ceiling at attestation time.

Status

Resolved

Medium

[M-01] KiraPayGateway#fastFinishVaultTransfer, fastFinishInvoice - Cross-chain fast-finisher permanently loses fronted funds when source vault is refunded

Description

`fastFinishVaultTransfer` has no requirement that `processVaultTransfer` was already called on the source chain, so a relayer can front delivery on the destination chain before any bridge transfer exists. If the source vault then expires and is refunded, both recovery paths are blocked: `claimVaultTransfer` requires bridged funds that never arrived, and `processVaultTransfer` rejects expired vaults.

Vulnerability Details

`fastFinishVaultTransfer` sets the fast-finisher and pulls funds from the relayer with no check that a corresponding bridge transfer was initiated on the source chain:

```
// fastFinishVaultTransfer - no check that processVaultTransfer was called on source
chain
landingFastFinisher[landingAddr] = msg.sender;
TokenUtils.transferFrom(vaultParams.toToken, msg.sender, address(this),
fastFinishParams.amountToDeliver);
```

`claimVaultTransfer` requires a non-zero bridged balance at the landing:

```
uint256 bridgedAmount = IKiraBridgeLanding(payable(landingAddr)).pull(deliverToken);
require(bridgedAmount > 0, NothingToClaim());
```

The same vulnerability exists in the invoice flow. `fastFinishInvoice` has no guard that `processInvoice` was called on the source chain. If the invoice expires without being processed, `claimInvoice` fails on `require(availableAmount > 0, NothingToClaim())` and destination-chain `refundInvoice` is blocked by `require(invoiceFastFinisher[invoiceHash] == address(0), InvoiceAlreadyFastFinished())`.



Impact

A relayer who fast-finishes a cross-chain vault transfer permanently loses their fronted funds if the source vault expires and is refunded before `processVaultTransfer` is called.

Recommendation

Relayers must confirm `processVaultTransfer` is included in a finalized source-chain block before calling `fastFinishVaultTransfer` on the destination chain. Similarly, they must confirm `processInvoice` is included in a finalized source-chain block before calling `fastFinishInvoice` on the destination chain.

Status

Acknowledged

[M-02] KiraPayGateway#claimInvoice - Insufficient bridged amount permanently blocks fast-finisher repayment and strands vault tokens

Description

When the bridge delivers fewer tokens to the invoice vault than `invoice.toAmount`, every branch of `claimInvoiceBridged` reverts. In the non-fast-finisher paths this leaves `refundInvoice` available as a recovery mechanism, but in the fast-finisher path the revert permanently blocks both `claimInvoice` and `refundInvoice`, leaving the fast-finisher with no way to recover the funds they already paid to the recipient.

Vulnerability Details

All three branches of `claimInvoiceBridged` assume `availableAmount >= invoice.toAmount` and revert when that assumption is violated. `availableAmount` is exactly what was swept from the invoice vault, whatever the bridge actually delivered, and can be less than `invoice.toAmount` due to a relayer supplying `bridgeTokenOut.amount < invoice.toAmount` in `processInvoice`, or bridge underdelivery.

Fast-finisher branch: `safeTransfer` reverts because the gateway only holds `availableAmount < invoice.toAmount`:

```
// KiraGatewayLib.sol
TokenUtils.transfer(params.bridgeTokenOut.token, payable(params.fastFinisher),
invoice.toAmount);
```

No fast-finisher, with swap: `_handleSwapAndFinalCall` reverts at the `InsufficientAmount` guard when `availableAmount < swapRoute.amountIn`, and additionally enforces `minSwapAmountOut = invoice.toAmount` which cannot be met from a smaller input:

```
uint256 minSwapAmountOut = invoice.toAmount;
```

No fast-finisher, no swap: fee collection and delivery both operate on `invoice.toAmount`, causing the delivery transfer to fail when `availableAmount < invoice.toAmount`:



```
uint256 netAmount = collectFeeFromGateway(cfg, invoice.toToken, invoice.toAmount,
true, accountId);
finalCallSuccess = deliver(cfg, invoice.toToken, recipient, finalCallData,
netAmount);
```

In all three cases the transaction reverts. For the non-fast-finisher paths this is recoverable, `invoiceFastFinisher` is `address(0)`, so `refundInvoice` remains unblocked and the relayer can retry with corrected parameters or the payer can reclaim funds after expiry.

The fast-finisher path is the critical case. `invoiceFastFinisher[invoiceHash]` was set by the earlier `fastFinishInvoice` call and is never rolled back, so `refundInvoice` on the destination chain also fails:

```
// KiraPayGateway.sol
require(invoiceStates[invoiceHash] == InvoiceState.UNSEEN, ...); // passes
require(invoiceFastFinisher[invoiceHash] == address(0), ...);    // reverts,
permanently blocked
```

Impact

Any bridge underdelivery or relayer misconfiguration of `bridgeTokenOut.amount` in `processInvoice` permanently locks the bridged tokens in the invoice vault and blocks the fast-finisher from recovering the funds they already paid to the recipient, with no on-chain recovery path.

Recommendation

Cap repayment in the fast-finisher branch at the lesser of `availableAmount` and `invoice.toAmount` so the transfer never exceeds what the gateway actually holds from this invoice:

```
uint256 repayAmount = availableAmount < invoice.toAmount ? availableAmount :
invoice.toAmount;
TokenUtils.transfer(params.bridgeTokenOut.token, payable(params.fastFinisher),
repayAmount);
```

```
paymentTokenConsumed = repayAmount;
```

Add an input guard in `processInvoice` to prevent relayers from supplying a bridge output below the invoice target. This comparison is only valid when the bridge token equals the destination token (no destination-chain swap); when a swap route is provided the minimum bridge amount is exchange-rate dependent and cannot be enforced with a simple check:

```
if (address(params.bridgeTokenOut.token) == address(invoice.toToken)) {
    require(params.bridgeTokenOut.amount >= invoice.toAmount,
    InsufficientBridgeOutput());
}
```

Finally, allow `refundInvoice` to proceed on the destination chain when the invoice is `UNSEEN` but `invoiceFastFinisher` is non-zero, so expiry provides a fallback recovery path for stranded vault tokens. Remove the hard block and route the vault balance to the fast-finisher as repayment:

```
// KiraPayGateway.sol - destination / same-chain branch
require(invoiceStates[invoiceHash] == InvoiceState.UNSEEN,
InvoiceAlreadyProcessed());
// Remove this line:
// require(invoiceFastFinisher[invoiceHash] == address(0),
InvoiceAlreadyFastFinished());
// If a fast-finisher is owed repayment, send vault balance to them instead of
refundAddress
address refundTo = invoiceFastFinisher[invoiceHash] != address(0)
    ? invoiceFastFinisher[invoiceHash]
    : invoice.refundAddress;
_createAndSweepInvoice(invoice, token, payable(refundTo));
```

Status

Resolved

[M-03] KiraPayGateway#fastFinishVaultTransfer - Fast-finishing an expired vault permanently locks relayer funds

Description

`fastFinishVaultTransfer` has no vault-expiry check, unlike `processVaultTransfer` which guards against it. A relayer can front delivery on an already-expired vault with no on-chain recovery path, since `processVaultTransfer`, the only function that would repay the fast-finisher, rejects expired vaults.

Vulnerability Details

`processVaultTransfer` enforces an expiry check; `fastFinishVaultTransfer` does not:

```
// processVaultTransfer
require(!isVaultExpired(vaultParams), VaultExpired());
// fastFinishVaultTransfer - expiry guard absent
_validateVaultAttestation(vaultParams, fastFinishParams.attestation);
require(vaultParams.toChainId == block.chainid, WrongChain());
// missing: require(!isVaultExpired(vaultParams), VaultExpired());
landingFastFinisher[landingAddr] = msg.sender;
TokenUtils.transferFrom(vaultParams.toToken, msg.sender, address(this),
fastFinishParams.amountToDeliver);
```

A relayer that calls `fastFinishVaultTransfer` on an already-expired vault delivers `amountToDeliver` to the recipient and sets `landingFastFinisher[landingAddr]`. When they attempt recovery via `processVaultTransfer`, the call reverts with `VaultExpired`. The recipient receives the delivery for free and the relayer has no recourse.

Impact

A relayer who calls `fastFinishVaultTransfer` on an expired vault permanently loses their fronted funds with no recovery path, as `processVaultTransfer` rejects expired vaults and no alternative repayment mechanism exists.

Recommendation

Add `require(!isVaultExpired(vaultParams), VaultExpired())` to `fastFinishVaultTransfer`, matching the guard already present in `processVaultTransfer`.

Status

Resolved

[M-04] KiraPayGateway#refundVault - Vault can be drained while a fast-finisher is owed repayment

Description

`refundVault` has no guard against sweeping vault funds when a fast-finisher has already fronted delivery for a same-chain vault transfer, leaving the relayer with no on-chain recovery path. The analogous protection exists in `refundInvoice` but was not carried over to vaults.

Vulnerability Details

`refundInvoice` blocks the refund path when a fast-finisher is pending:

```
// refundInvoice - guard present
require(invoiceFastFinisher[invoiceHash] == address(0),
InvoiceAlreadyFastFinished());
```

`refundVault` has no equivalent check and sweeps unconditionally once the vault is expired:

```
require(isVaultExpired(params), VaultNotExpired());
// no check: require(landingFastFinisher[landingAddr] == address(0), ...)
reusableVault.sweep(params, tokens[i], payable(params.refundAddress), 0);
```

A relayer fast-finishes a live vault, delivering `amountToDeliver` of `toToken` to the recipient from their own balance and setting `landingFastFinisher[landingAddr] = relayer`. The vault then expires naturally before `processVaultTransfer` is called. Anyone, including the payer acting as `refundAddress`, calls `refundVault`, sweeping the full vault balance to `refundAddress`. The relayer has no remaining recovery path: `processVaultTransfer` reverts with `VaultExpired`.

Impact

A relayer who fast-finishes a same-chain vault transfer permanently loses their fronted funds when `refundVault` drains the vault before `processVaultTransfer` is called, with the payer's original deposit going to `refundAddress` instead of repaying the relayer.

Recommendation

Mirror the `refundInvoice` pattern in `refundVault`: derive the active landing address for the given vault params and revert if `landingFastFinisher[landingAddr] != address(0)`.

Status

Resolved

[M-05] KiraGatewayLib#claimInvoiceBridged - Fast-finisher repaid in bridge token instead of invoice token

Description

When a relayer fast-finishes a cross-chain invoice, it fronts `invoice.toToken`. When the bridge lands on the destination chain and `claimInvoice` is called, the fast-finisher repayment uses `params.bridgeTokenOut.token`, the token the bridge delivered, not necessarily `invoice.toToken`. These two tokens differ whenever the bridge delivers a bridgeable variant (e.g. USDC.e) while the invoice requested the canonical token (e.g. USDC).

Vulnerability Details

In `fastFinishInvoice`, the relayer fronts `invoice.toToken` at `invoice.toAmount`. When `claimInvoice` runs on the destination chain, the protocol repays the fast-finisher in `params.bridgeTokenOut.token`:

```
TokenUtils.transfer(params.bridgeTokenOut.token, payable(params.fastFinisher),  
invoice.toAmount);
```

There is no check that `params.bridgeTokenOut.token == invoice.toToken` in this branch. The no-swap, no-fast-finisher branch does enforce this equality, but the fast-finisher repayment path does not. The mismatch is live in any cross-chain configuration where the bridge input and output token differ.

Impact

A relayer who fast-finishes a cross-chain invoice is repaid in the bridge-delivered token rather than the one it fronted, losing the spread between `invoice.toToken` and the bridge-delivered token on the full invoice amount.

Recommendation

Change the repayment to transfer `invoice.toToken` rather than `params.bridgeTokenOut.token`. This eliminates the mismatch entirely. If the

bridge-delivered token is to be used for repayment, add an explicit equality check and revert if `params.bridgeTokenOut.token != invoice.toToken`.

Status

Resolved



[M-06] KiraSwapExecutor#swapAndTransfer - Fast-finisher captures swap surplus from invoice settlement

Description

When settling a same-chain invoice with a swap and a fast-finisher, `swapAndTransfer` sends the fast-finisher both the owed repayment (`invoice.toAmount`) and all swap output above it. The payer's `toToken` upside from a favourable swap accrues to the fast-finisher rather than `invoice.refundAddress`.

Vulnerability Details

`processInvoiceSameChain` sets `deliveryTarget` to the fast-finisher and clears `callData` to empty bytes when a fast-finisher is recorded:

```
address deliveryTarget = params.fastFinisher != address(0) ? params.fastFinisher :
invoice.recipientAddress;
bytes memory callData = params.fastFinisher != address(0) ? bytes("") :
invoice.finalCallData;
```

`_handleSwapAndFinalCall` is called with `collectFee = false` and `minSwapAmountOut = invoice.toAmount`. In `swapAndTransfer`, the invoice no-final-call branch transfers `invoice.toAmount` and then sweeps the remaining balance, both going to the same recipient:

```
TokenUtils.transfer(params.route.tokenOut, payable(params.recipient),
amountToProcess);
TokenUtils.transferBalance({token: params.route.tokenOut, recipient:
payable(params.recipient)});
```

`params.recipient` is the fast-finisher. The swap may produce more than `invoice.toAmount` of `toToken`, any such surplus is swept to the fast-finisher along with the repayment. The gateway returns the payer's `paymentToken` surplus to `invoice.refundAddress`, but the `toToken` swap surplus has no equivalent return path.

Impact

The payer loses the `toToken` upside from a favourable swap, and that value accrues to the fast-finisher rather than the invoice refund address.

Recommendation

In the invoice fast-finisher repayment path of `swapAndTransfer`, transfer only `invoice.toAmount` to the fast-finisher and route any remaining `tokenOut` balance separately, either via a surplus address threaded through the swap params, or by returning it to the gateway for forwarding to `refundAddress`.

Status

Resolved

[M-07] KiraGatewayLib#processVaultTransferCrossChain - Cross-decimal collateral guard breaks vault cross-chain settlements

Description

The no-swap branch of `processVaultTransferCrossChain` validates vault collateral by comparing a source-chain token amount against a destination-chain token amount, which is only correct when both chains use the same decimal precision for the token.

Vulnerability Details

When a vault transfer takes the no-swap cross-chain path, the collateral check compares two quantities measured in different units:

```
// KiraGatewayLib.sol
require(
    processParams.availableAmount > processParams.bridgeTokenOut.amount,
    IKiraPayGateway.InsufficientAmount()
);
```

`availableAmount` is denominated in the source chain's token decimals, `bridgeTokenOut.amount` is denominated in the destination chain's token decimals. When the same logical asset uses different decimal precisions across chains, USDC is 6 decimals on Ethereum and 18 decimals on certain L2s, the comparison breaks in both directions. On a 6→18 route, is always false, so every settlement attempt reverts regardless of vault balance. On an 18→6 route, the magnitude difference trivially satisfies the guard, making it ineffective as a collateral check.

`ProcessInvoiceParams` solves this correctly by including a `bridgeAmountIn` field, the source-side amount computed by the Across API, which the invoice path uses to compare:

```
require(params.availableAmount >= inAmount, IKiraPayGateway.InsufficientAmount());
```

`ProcessVaultTransferParams` has no equivalent field, leaving the vault cross-chain no-swap path without a source-side amount for the collateral check. Across V3

supports mismatched decimals via `allowUnmatchedDecimals`, so this route configuration is expected in production.

Impact

Vault cross-chain settlements are permanently blocked on any token route where source and destination decimals differ, and the collateral check provides no protection on routes where the destination has fewer decimals than the source.

Recommendation

Add a `bridgeAmountIn` field to `ProcessVaultTransferParams` (mirroring `ProcessInvoiceParams.bridgeAmountIn`) and replace the cross-unit comparison.

Status

Resolved

Low

[L-01] KiraPayGateway - Missing event emissions on admin setter functions

Description

`setPricer` and `setDefaultSlippageBps` update state variables without emitting events, making the changes invisible to off-chain consumers.

Vulnerability Details

`setPricer` replaces the `pricer` address with no event emission.

```
function setPricer(IKiraPayPricer _pricer) external onlyOwner {
    require(address(_pricer) != address(0), InvalidRecipient());
    pricer = _pricer;
}
```

`setDefaultSlippageBps` updates the global slippage parameter with no event emission.

```
function setDefaultSlippageBps(uint256 _slippageBps) external onlyOwner {
    require(_slippageBps <= 10_000, SlippageTooHigh());
    defaultSlippageBps = _slippageBps;
}
```

Note that the adjacent `setGuardian` and `setRelayer` functions do emit events, making the omission inconsistent.

Impact

Off-chain systems cannot detect or react to pricer replacements or slippage parameter changes, undermining monitoring and protocol transparency.

Recommendation

Declare and emit a `PricerSet` event that logs both the old and new pricer addresses, and a `DefaultSlippageBpsSet` event that logs both the old and new slippage values.

Status

Resolved



[L-02] KiraGatewayLib#processVaultTransferCrossChain - Strict inequality rejects equal-amount vault settlements

Description

In the no-swap cross-chain vault path, the available amount must be strictly greater than the bridge output amount. The analogous invoice path uses `>=`, making this an asymmetric and unnecessarily restrictive guard.

Vulnerability Details

In `processVaultTransferCrossChain`, the no-swap path enforces:

```
require(  
    processParams.availableAmount > processParams.bridgeTokenOut.amount,  
    IKiraPayGateway.InsufficientAmount()  
);
```

The corresponding check in `processInvoiceCrossChain` uses `>=`:

```
require(params.availableAmount >= inAmount, IKiraPayGateway.InsufficientAmount());
```

The strict `>` means a vault deposit that exactly equals `bridgeTokenOut.amount` reverts, even when the bridge protocol charges zero fees. There is no documented requirement that the available amount must exceed the bridge output by any positive margin.

Impact

Vault transfers where the deposited amount exactly equals the bridge output amount fail to process, forcing payers to over-deposit to satisfy a constraint that does not exist in the invoice path.

Recommendation

Change the strict inequality to `>=`:

```
require(  
    processParams.availableAmount >= processParams.bridgeTokenOut.amount,
```

```
IKiraPayGateway.InsufficientAmount()  
);
```

Status

Resolved



[L-03] KiraPayGateway#fastFinishInvoice - Wrong fee tier applied on fast-finish delivery

Description

`fastFinishInvoice` applies the routed fee tier (`routedFeeBps`) to what is a direct token delivery, when no swap or bridge is involved on the delivery side.

Vulnerability Details

The protocol defines two fee tiers: `baseFeeBps` for direct token delivery (same-chain, no swap) and `routedFeeBps` for payments that involve a swap, bridge, or both. `fastFinishInvoice` in the same chain is a direct transfer, the relayer deposits `invoice.toAmount` of `invoice.toToken` and the gateway immediately sends `netAmount` to `invoice.recipientAddress` with no swap and no bridge hop.

Despite this, the fee is always collected with `isRouted = true`:

```
uint256 netAmount = cfg.collectFeeFromGateway(invoice.toToken, invoice.toAmount,
true, invoice.accountId);
```

`collectFeeFromGateway` passes `isRouted` directly to `feeManager.computeFee`, which selects `routedFeeBps` when `true` and `baseFeeBps` when `false`.

Impact

Recipients of fast-finished same chain invoices are charged the higher routed fee rate instead of the base rate, causing them to receive less than they would from an equivalent direct settlement.

Recommendation

Pass `false` for `isRouted` in `fastFinishInvoice` when the invoice is on the same chain.

Status

Resolved

[L-04] KiraPayGateway#processInvoice - Swap surplus stranded at gateway in cross-chain invoice flow

Description

When processing a cross-chain invoice with a swap, any favorable slippage accumulates at the gateway permanently, the bridge sends only the minimum configured output and no recovery path exists for the difference.

Vulnerability Details

In `KiraSwapExecutor.swapAndTransfer`, the invoice path sets the return value to `minSwapAmountOut`, then transfers both the minimum and any surplus to the recipient:

```
amountToProcess = params.minSwapAmountOut;
TokenUtils.transfer(params.route.tokenOut, payable(params.recipient),
amountToProcess);
TokenUtils.transferBalance({token: params.route.tokenOut, recipient:
payable(params.recipient)});
return (amountToProcess, true);
```

For cross-chain payments, `params.recipient` is the gateway. Both transfers land at the gateway, so the gateway holds the full swap output. `_handleSwapAndFinalCall` returns only `minSwapAmountOut` as `amountProcessed`, and `_bridgeOut` bridges only that amount:

```
(amountProcessed, _) = _handleSwapAndFinalCall(...);
_bridgeOut(cfg, ..., bridgeToken, amountProcessed);
```

The surplus, the difference between actual swap output and `minSwapAmountOut`, stays in the gateway. The surplus refund at the end of `processInvoice` covers only the input `paymentToken`:

```
uint256 surplus = availableAmount - paymentTokenConsumed;
```

The bridge token surplus is never returned to the payer or refund address.

Impact

Favorable swap slippage from cross-chain invoice payments accumulates in the gateway with no recovery path

Recommendation

Bridge the full swap output rather than just `minSwapAmountOut`.

Status

Resolved

QA

[Q-01] All contracts - Unlocked Pragma Version

Description

The contract uses a floating pragma that accepts any Solidity compiler from 0.8.28 upward.

Vulnerability Details

The contracts declare `pragma solidity ^0.8.20;`, which does not bind compilation to a specific version. This allows contracts to be compiled with different compiler versions than those tested. Future compiler versions may introduce semantic changes, new defaults, or undiscovered bugs that alter contract behavior.

Impact

The contract may be compiled with an unintended Solidity version, introducing compiler level behavior changes that were not present during the audit.

Recommendation

Lock the pragma to the specific compiler version used during development, for example if 0.8.28 was used: `pragma solidity 0.8.28;`

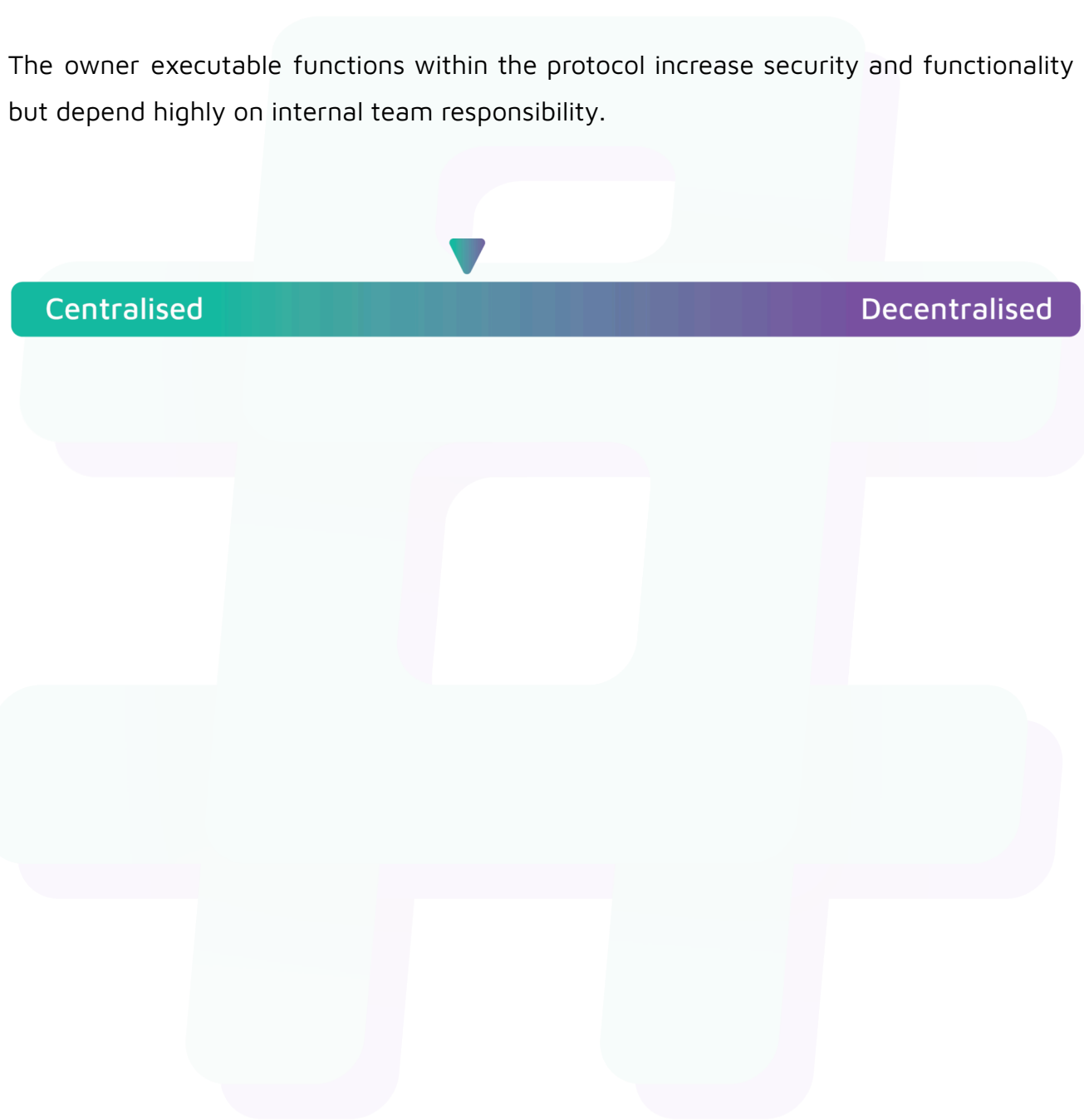
Status

Resolved

Centralisation

The KIRAPAY Deal project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the KIRAPAY Deal project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.